

# Clase 6:

# Programación

Manejo de archivos

# Temario

- Manejo de archivos
- Procesamiento de archivos
- Matrices y Operaciones matriciales.
- Otras estructuras: grafos, árboles.
- Requisitos de aprobación del curso
  - Cuestionario
  - Trabajo grupal

# Manejo de archivos

Existen dos formas básicas de acceder a un archivo:

- Una es **utilizarlo como un archivo de texto**, que procesaremos **línea por línea**
- La otra es tratarlo **como un archivo binario**, que procesaremos **byte por byte**.

• • • • • • • • • •

# Manejo de archivos

Para **abrir un archivo** usaremos la función **open**:

```
archivo = open("archivo.txt")
```

Para procesarlo línea por línea, es posible hacerlo de la siguiente forma:

```
linea=archivo.readline()
```

```
while linea != "":
```

```
    # procesar linea
```

```
    linea=archivo.readline()
```

O usando

```
for linea in archivo:
```

```
    # procesar línea
```

# Manejo de archivos

Para **obtener todas las líneas** del archivo utilizando una sola llamada a función:

```
lineas = archivo.readlines()
```

En este caso, la variable **lineas** tendrá una lista de **cadenas** con todas las líneas del archivo.

# Manejo de archivos

## Cerrar un archivo

Al terminar de trabajar con un archivo, es recomendable cerrarlo, por diversos motivos:

- En algunos sistemas los archivos **sólo pueden ser abiertos de a un programa por vez**
- **Lo que se haya escrito no se guardará** realmente hasta no cerrar el archivo

Para **cerrar un archivo** simplemente se debe llamar a:  
**archivo.close()**

# Manejo de archivos

Para **mostrar todas las líneas de un archivo**, precedidas por el número de línea, podemos hacerlo como en el Código:

```
archivo = open("archivo.txt")
i = 1
for linea in archivo:
    linea = linea.rstrip("\n")
    print " %4d: %s" % (i, linea)
    i+=1
archivo.close()
```

# Manejo de archivos

## Modo de apertura de archivos:

La función **open** recibe un **parámetro opcional**. Los **tres modos de apertura** que se pueden especificar son:

- Modo de **sólo lectura ('r')**. En este caso no es posible realizar modificaciones sobre el archivo, solamente leer su contenido.
- Modo de **sólo escritura ('w')**. En este caso el archivo se sobre escribe (vaciado) si existe, y se lo crea si no existe.
- Modo **sólo escritura posicionándose al final del archivo (a)**. En este caso se crea el archivo, si no existe, pero en caso de que exista **se posiciona al final**, manteniendo el contenido original.

# Manejo de archivos

## Escribir un archivo:

De la misma forma que para la lectura, existen **dos formas** distintas de escribir a un archivo:

### Mediante cadenas:

- `archivo.write(cadena)`

### O mediante listas de cadenas:

- `archivo.writelines(lista_de_cadenas)`

Será necesario **agregar los caracteres de fin de línea** a las cadenas que se vayan a escribir en el archivo, el `\n`

# Manejo de archivos

## Ejemplo:

```
• saludo = open("saludo.py", "w")  
  saludo.write("""print "Hola Mundo" """)  
• saludo.close()
```

# Manejo de archivos

## Agregar información a un archivo

**Abrir un archivo en modo agregar al final** es útil para **escribir** un archivo de bitácora (o **archivo de log**), que nos permita ver los distintos eventos que se fueron sucediendo

# Manejo de archivos – crear el archivo log.py

```
#!/usr/bin/env python
```

```
# encoding: latin1
```

```
# El módulo datetime se utiliza para obtener la fecha y hora actual.
```

```
import datetime
```

```
def abrir_log(nombre_log):
```

```
    """ Abre el archivo de log indicado. Devuelve el archivo abierto.
```

```
    Pre: el nombre corresponde a un nombre de archivo válido.
```

```
    Post: el archivo ha sido abierto posicionándose al final. """
```

```
    archivo_log = open(nombre_log, "a")
```

```
    guardar_log(archivo_log, "Iniciando registro de errores")
```

```
    return archivo_log
```

```
def guardar_log(archivo_log, mensaje):
```

```
    """ Guarda el mensaje en el archivo de log, con la hora actual.
```

```
    Pre: el archivo de log ha sido abierto correctamente.
```

```
    Post: el mensaje ha sido escrito al final del archivo. """
```

```
    hora_actual = str(datetime.datetime.now()) # Obtiene la hora actual
```

```
    archivo_log.write(hora_actual+" "+mensaje+"\n") # Guarda mensaje
```

```
def cerrar_log(archivo_log):
```

```
    """ Cierra el archivo de log.
```

```
    Pre: el archivo de log ha sido abierto correctamente.
```

```
    Post: el archivo de log se ha cerrado. """
```

```
    guardar_log(archivo_log, "Fin del registro de errores")
```

```
    archivo_log.close()
```

# Manejo de archivos – en formato binario

Existen **archivos** en los que cada **byte** tiene un **significado** particular, y es necesario manipularlos conociendo el formato en que están los datos

Para **abrir un archivo** y manejarlo de **forma binaria** es necesario **agregarle una 'b' al parámetro de modo**.

Para procesar el archivo de a bytes se utiliza la función ***contenido = archivo.read(n)*** para **leer n bytes** y se utiliza ***archivo.write(contenido)***, para **escribir contenido en la posición actual** del archivo.

Para obtener la **posición actual** se utiliza ***archivo.tell()***, indica la cantidad de bytes desde el comienzo del archivo

# Manejo de archivos – Persistencia de datos

Se llama **persistencia** a la **capacidad de guardar la información de un programa** para poder volver a utilizarla en otro momento.

supongamos que en el desarrollo **de un juego se quiere guardar en un archivo la información referente a los ganadores**, el puntaje máximo obtenido y el tiempo de juego en el que obtuvieron ese puntaje.

En el juego, esa información podría estar almacenada en una lista de tuplas:

**[(nombre1, puntaje1, tiempo1), (nombre2, puntaje2, tiempo2), ...]**

En este caso, **para facilitar la lectura del archivo de puntajes** para los humanos, se **decide guardarlos en un archivo de texto**, donde cada tupla ocupará una línea y los valores de las tuplas estarán separados por comas.

# Manejo de archivos – puntaje.py

# El módulo para guardar puntajes en archivo.

```
def guardar_puntajes(nombre_archivo, puntajes):  
    """ Guarda la lista de puntajes en el archivo.  
    Pre: nombre_archivo corresponde a un archivo válido,  
    puntajes corresponde a una lista de tuplas de 3 elementos.  
    Post: se guardaron los valores en el archivo,  
    separados por comas.  
    """  
    archivo = open(nombre_archivo, "w")  
    for nombre, puntaje, tiempo in puntajes:  
        archivo.write(nombre+", "+str(puntaje)+", "+tiempo+"\n")  
    archivo.close()
```

```
def recuperar_puntajes(nombre_archivo):  
    """ Recupera los puntajes a partir del archivo provisto.  
    Devuelve una lista con los valores de los puntajes.  
    Pre: el archivo contiene los puntajes en el formato esperado,  
    separados por comas  
    Post: la lista devuelta contiene los puntajes en el formato:  
    [(nombre1,puntaje1,tiempo1),(nombre2,puntaje2,tiempo2)].  
    """
```

# Manejo de archivos – continuación de puntaje.py

**# Continuación de módulo para guardar puntajes** en archivo.

```
puntajes = []
archivo = open(nombre_archivo, "r")
for linea in archivo:
    nombre, puntaje, tiempo = linea.rstrip("\n").split(",")
    puntajes.append((nombre,int(puntaje),tiempo))
archivo.close()
return puntajes

#Fin de archivo puntaje.py
```

## Ejemplo de uso:

```
>>> valores = [("Pepe", 108, "4:16"), ("Juana", 2315, "8:42")]
>>> puntajes.guardar_puntajes("puntajes.txt", valores)
>>> recuperado = puntajes.recuperar_puntajes("puntajes.txt")
>>> print recuperado
```

```
[('Pepe', 108, '4:16'), ('Juana', 2315, '8:42')]
```

# Manejo de archivos –Archivos CSV

Un formato que **suele usarse para transferir datos entre programas** es **csv** (del inglés **comma separated values**: valores separados por comas) es un formato bastante sencillo:

**Nombre,Apellido,Telefono,Cumpleaños**

**"John","Smith","555-0101","1973-11-24"**

**"Jane","Smith","555-0101","1975-06-12"**

En el ejemplo es una pequeña base de datos. En la primera línea del archivo están los **nombres de los campos**, un dato que facilita entender el archivo.

En las siguientes líneas se ingresan los datos de la base de datos, cada campo separado por comas.

# Matrices

Una **matriz** es una **estructura de datos bidimensional** que **consta de filas y columnas** de elementos numéricos dispuestos en forma rectangular.

Las matrices **son utilizadas** en programación **para representar datos tabulares, imágenes, grafos ponderados** y mucho más.

# Matrices

Es común **usar listas anidadas para representar matrices**.  
Por ejemplo, la matriz:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

puede ser representada como:

```
>>> matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

La matriz es una **lista con tres elementos, siendo cada elemento una fila de la de la matriz**.

Podemos elegir **una fila entera de la matriz** de la forma normal:

```
>>> matriz[1]  
[4, 5, 6]
```

# Matrices

Ejemplo de **Matriz en Python:**

```
# Definir una matriz de 2x3
matriz = [
    [1, 2, 3],
    [4, 5, 6]
]
```

```
# Acceder a un elemento específico de la matriz
print(matriz[0][1]) # Salida: 2
```

# Operaciones con Matrices

**Operaciones con matrices utilizando listas anidadas y operaciones de bucles.**

**Suma de Matrices:** Sumar dos matrices elemento por elemento utilizando bucles.

**Resta de Matrices:** Similar a la suma, puedes restar dos matrices elemento por elemento.

**Multiplicación Escalar:** Multiplicar cada elemento de la matriz por el escalar.

# Operaciones con Matrices

## Suma de Matrices:

```
def suma_matrices(matriz1, matriz2):  
    resultado = []  
    for i in range(len(matriz1)):  
        fila = []  
        for j in range(len(matriz1[0])):  
            fila.append(matriz1[i][j] + matriz2[i][j])  
        resultado.append(fila)  
    return resultado
```

# Ejemplo de uso

```
matriz1 = [[1, 2], [3, 4]]
```

```
matriz2 = [[5, 6], [7, 8]]
```

```
resultado = suma_matrices(matriz1, matriz2)
```

```
print(resultado) # Salida: [[6, 8], [10, 12]]
```

# Operaciones con Matrices

## Multiplicación Escalar:

```
def multiplicacion_escalar(matriz, escalar):  
    resultado = []  
    for i in range(len(matriz)):  
        fila = []  
        for j in range(len(matriz[0])):  
            fila.append(matriz[i][j] * escalar)  
        resultado.append(fila)  
    return resultado
```

# Ejemplo de uso

```
matriz = [[1, 2], [3, 4]]
```

```
escalar = 2
```

```
resultado = multiplicacion_escalar(matriz, escalar)
```

```
print(resultado) # Salida: [[2, 4], [6, 8]]
```

# Operaciones Adicionales con NumPy

**NumPy** es una **biblioteca para procesamiento numérico** en Python que proporciona muchas **funciones y métodos para operar con matrices** de manera eficiente. Además de la suma de matrices, NumPy permite realizar una amplia gama de operaciones, como:

**Producto Punto (Dot Product):** `np.dot()` para calcular el producto punto de dos matrices.

**Multiplicación de Matrices:** `np.matmul()` para multiplicar dos matrices.

**Transposición de Matrices:** `np.transpose()` para obtener la transpuesta de una matriz.

**Inversa de una Matriz:** `np.linalg.inv()` para calcular la inversa de una matriz.

# Operaciones Adicionales con NumPy

Primero se debe **instalar** la librería con:

- **pip install numpy**

Una vez instalada, se debe **importar** para luego usar:

```
import numpy as np
```

```
# Producto Punto (Dot Product)
```

- **matriz1 = np.array([[1, 2], [3, 4]])**

```
matriz2 = np.array([[5, 6], [7, 8]])
```

```
resultado_dot = np.dot(matriz1, matriz2)
```

```
print(resultado_dot)
```

```
# Multiplicación de Matrices
```

```
resultado_matmul = np.matmul(matriz1, matriz2)
```

```
print(resultado_matmul)
```

# Operaciones Adicionales con NumPy

```
# Transposición de Matrices
```

```
matriz_transpuesta = matriz1.T
```

```
print(matriz_transpuesta)
```

```
# Inversa de una Matriz
```

```
matriz_inversa = np.linalg.inv(matriz1)
```

```
print(matriz_inversa)
```

**Las operaciones con NumPy son más concisas y eficientes** en términos de código.

# Otras Estructuras: Grafos y Árboles

## Grafos:

Un grafo es una **estructura de datos** que consta de un conjunto de **nodos (vértices)** y un conjunto de **aristas** que **conectan pares de nodos**. Los **grafos se utilizan para modelar relaciones entre entidades** y se pueden dirigir (con aristas con direcciones) o no dirigidos.

## Ejemplo:

- # Implementación de un grafo

```
grafo = {  
    'A': ['B', 'C'],  
    'B': ['A', 'C', 'D'],  
    'C': ['A', 'B', 'D'],  
    'D': ['B', 'C']  
}
```

**Las claves del diccionario** representan los **nodos** del grafo, y **los valores asociados** a cada clave representan los **nodos conectados a ese nodo**.

# Otras Estructuras: Grafos y Árboles

**NetworkX** es una biblioteca de Python diseñada para la creación y manipulación de estructuras complejas de redes o grafos. Es una herramienta poderosa y flexible que permite trabajar con grafos dirigidos y no dirigidos, multigrafos, grafos ponderados y no ponderados, entre otros tipos de grafos.

Primero se debe **instalar** la librería con:

```
pip install networkx
```

Una vez instalada, se debe **importar** para luego usar:

```
import networkx as nx
```

# Otras Estructuras: Grafos y Árboles

## Ejemplo de uso:

```
• import networkx as nx • • • • • • •  
• # Crear un grafo • • • • • • •  
G = nx.Graph()  
•  
• # Agregar nodos • • • • • • •  
G.add_nodes_from(['A', 'B', 'C', 'D'])  
• • • • • • •  
• # Agregar aristas • • • • • • •  
G.add_edges_from([('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D'), ('C', 'D')])  
•  
• # Dibujar el grafo • • • • • • •  
nx.draw(G, with_labels=True, node_color='lightblue',  
node_size=2000, font_size=20)
```

# Otras Estructuras: Grafos y Árboles

## Árboles:

Un árbol es una **estructura de datos jerárquica** que **consiste en un conjunto de nodos**, donde **uno de los nodos** se designa como **la raíz** y **el resto** de los nodos se organizan en **niveles o capas**. Cada nodo, excepto la raíz, está conectado por exactamente una arista a un nodo padre.

## Ejemplo:

# Implementación de un árbol

**class** Nodo:

**def** \_\_init\_\_(**self**, valor):

**self**.valor = valor

**self**.hijos = []

**def** agregar\_hijo(**self**, hijo):

**self**.hijos.append(hijo)

# Otras Estructuras: Grafos y Árboles

# Crear nodos

```
• raiz = Nodo('A')  
• nodo_b = Nodo('B')  
• nodo_c = Nodo('C')  
• nodo_d = Nodo('D')
```

# Construir el árbol

```
• raiz.agregar_hijo(nodo_b)  
raiz.agregar_hijo(nodo_c)  
nodo_b.agregar_hijo(nodo_d)
```

# Otras Estructuras: Grafos y Árboles

También podemos trabajar con la biblioteca **anytree** para trabajar con árboles.

Primero se debe **instalar** la librería con:

- **pip install anytree**

Una vez instalada, se debe **importar** para luego usar:

```
from anytree import Node, RenderTree
```

# Otras Estructuras: Grafos y Árboles

Ejemplo:

```
• from anytree import Node, RenderTree • • •  
  
• # Crear nodos • • • • •  
  raiz = Node("A")  
• nodo_b = Node("B", parent=raiz) • • •  
  nodo_c = Node("C", parent=raiz)  
• nodo_d = Node("D", parent=nodo_b) • • •  
  
  # Mostrar el árbol  
  for pre, fill, node in RenderTree(raiz):  
    print("%s%s" % (pre, node.name))
```

